Boring but important disclaimers:

- If you are not getting this from the GitHub repository or the associated Canvas page (e.g. CourseHero, Chegg etc.), you are probably getting the substandard version of these slides Don't pay money for those, because you can get the most updated version for free at

  `https://github.com/julianmak/OCES4303_ML_ocean`

  The repository principally contains the compiled products rather than the source for size reasons.
- Associated Python code (as Jupyter notebooks mostly) will be held on the same repository. The source data however might be big, so I am going to be naughty and possibly just refer you to where you might get the data if that is the case (e.g. JRA-55 data). I know I should make properly reproducible binders etc., but I didn't...
- I do not claim the compiled products and/or code are completely mistake free (e.g. I know I don't write Pythonic code). Use the material however you like, but use it at your own risk.
- As said on the repository, I have tried to honestly use content that is self made, open source or explicitly open for fair use, and citations should be there. If however you are the copyright holder and you want the material taken down, please flag up the issue accordingly and I will happily try and swap out the relevant material.

OCES 4303 :

an introduction to data-driven and ML methods in ocean sciences

Session 8: intro to neural networks

Outline

- anatomy of a neural network
  → nodes, weights, hidden layers, activation functions
  → back-propagation

- (multi-layer) perceptrons (MLPs)
  → simple perceptrons and interface with linear models
  → solvers, loss curves, various options

- cats and dogs example: classification and regression

Oceanic application

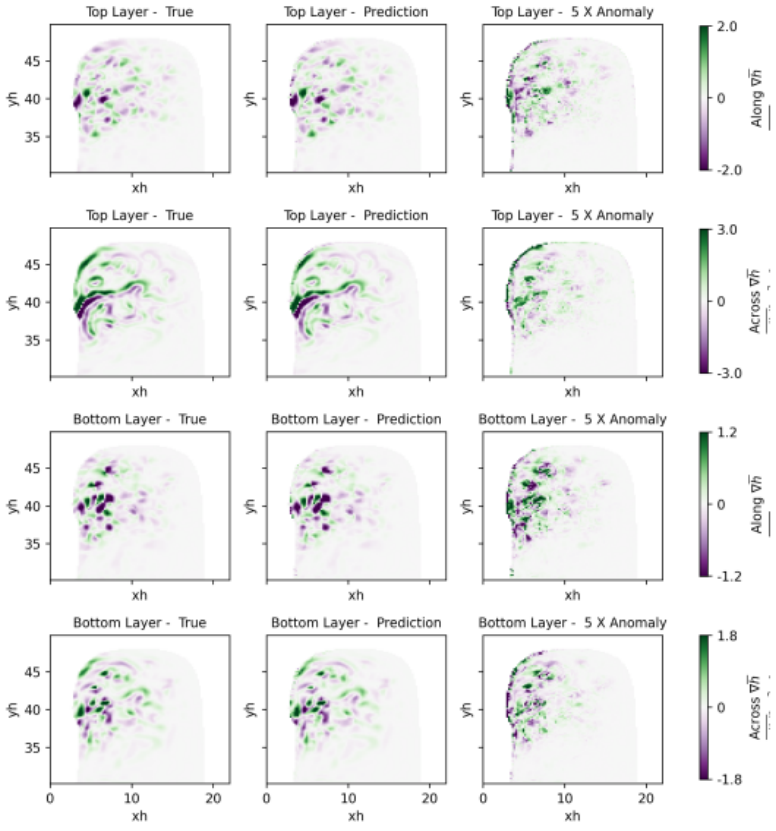


Figure: Model truth, predicted and mismatches in the predicted thickness fluxes from a double gyre calculation. From Balwada *et al.* (submitted).

- many instances where neural networks are used

  → but usually not in the MLP form for reasons to be elaborated

- example in predicting a spatially varying field, application in parameterisations

  → prediction here for **thickness** fluxes; see next sessions with CNNs for more

Recap: classifiers/regressors

- recall for supervised learning we basically have

$$Y = f(X),$$

  and given X and Y we want the classifer/regressor f

- f can be represented
  $\rightarrow$ symbolically (e.g. linear models)
  $\rightarrow$ as decision trees (e.g. random forests)
  $\rightarrow$ as a chain of elementary operations (e.g. neural networks)

Neural networks: anatomy

- easiest probably with an example:

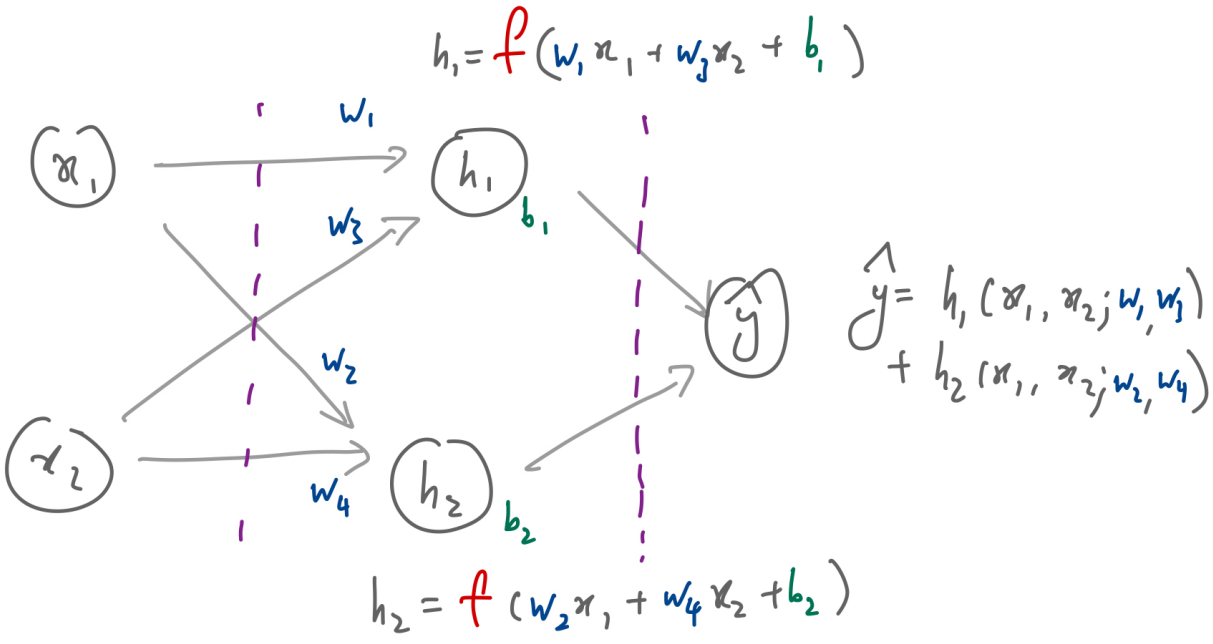


Figure: Made up example of a neural network (that you can code/train up yourself in principle; see notebook).

- input $X = (x_1, x_2)$ passes through network and gets transformed into prediction $Y = \hat{Y}$

Neural networks: anatomy

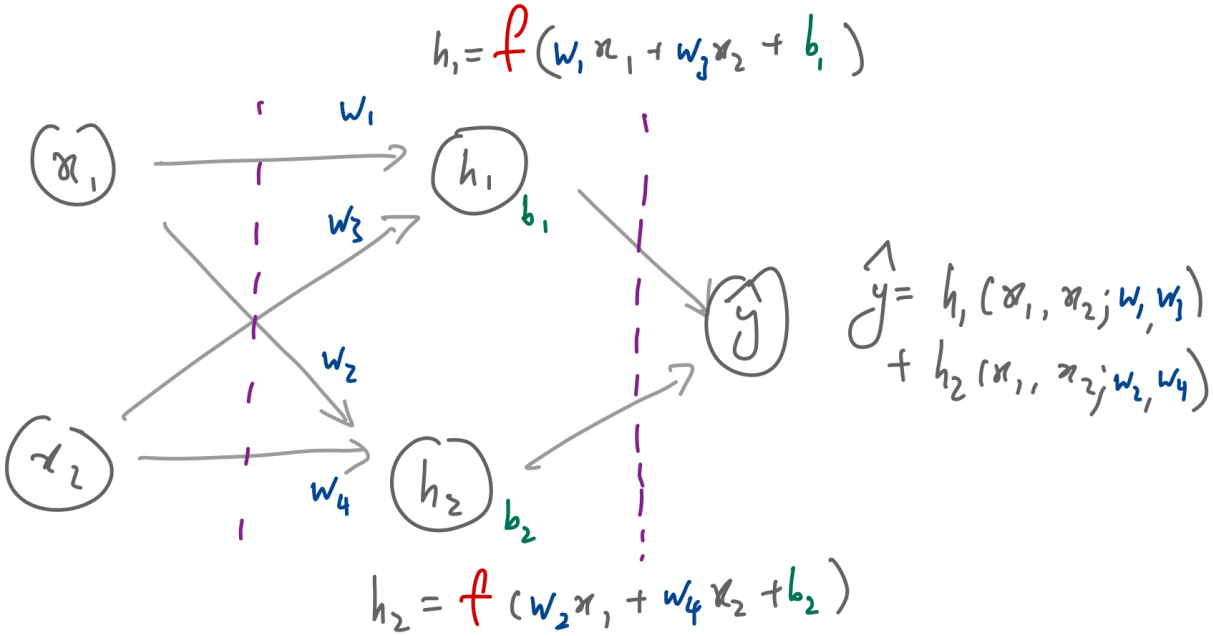


Figure: Made up example of a neural network (that you can code/train up yourself in principle; see notebook).

- each hidden layer have nodes, where we
  → multiply things by weights w_i
  → add on biases b_i
  → passed through an activation function f

Neural networks: anatomy

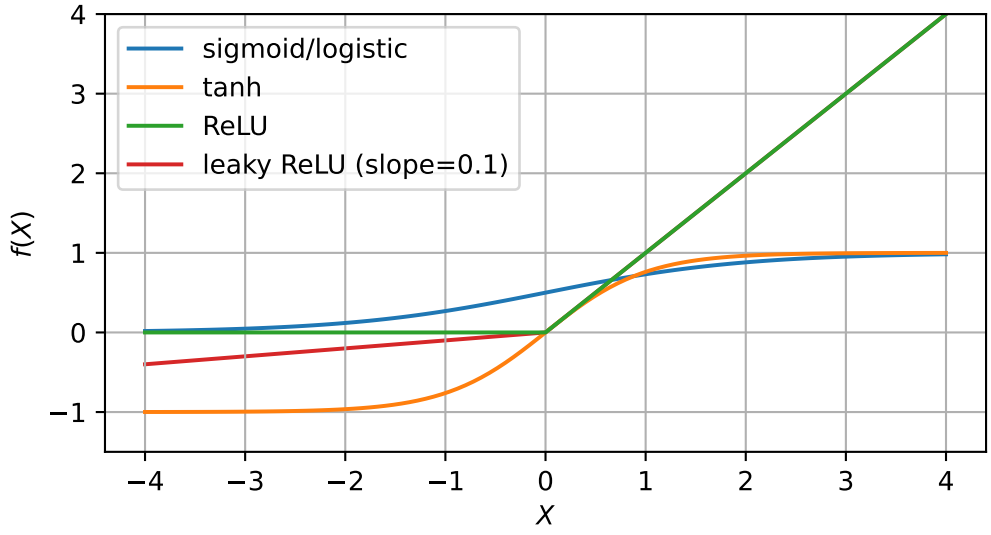


Figure: Samples of common activation functions: sigmoid (bounded between $[0, 1]$), tanh (bounded between $[-1, 1]$) and ReLU (bounded below by 0). Note ReLU is basically hinge loss but flipped the other way around. Leaky ReLU has a slope in the negative part.

- activation functions add nonlinearity to the system (see later)

  $\rightarrow$ maps the elementary operations to known ranges

  $\rightarrow$ usually piecewise differentiable (see three slides later for reason)

  $\rightarrow$ e.g. ReLU controls 'firing' of neurons (if $x < 0$)

Neural networks: as approximators

Q. just a bunch of elementary operations, cannot be complex enough surely?

- universal approximation theorem for neural networks:

  $\rightarrow$ 'sensible' (!?) functions can be approximated to using above operations given sufficient 'complexity'

  $\rightarrow$ 'complexity' means number of nodes and/or layers

- note: an **existence** theorem

  $\rightarrow$ doesn't tell you how you would train it

  $\rightarrow$ there are lower/upper bounds for complexity in some cases

Neural networks: training

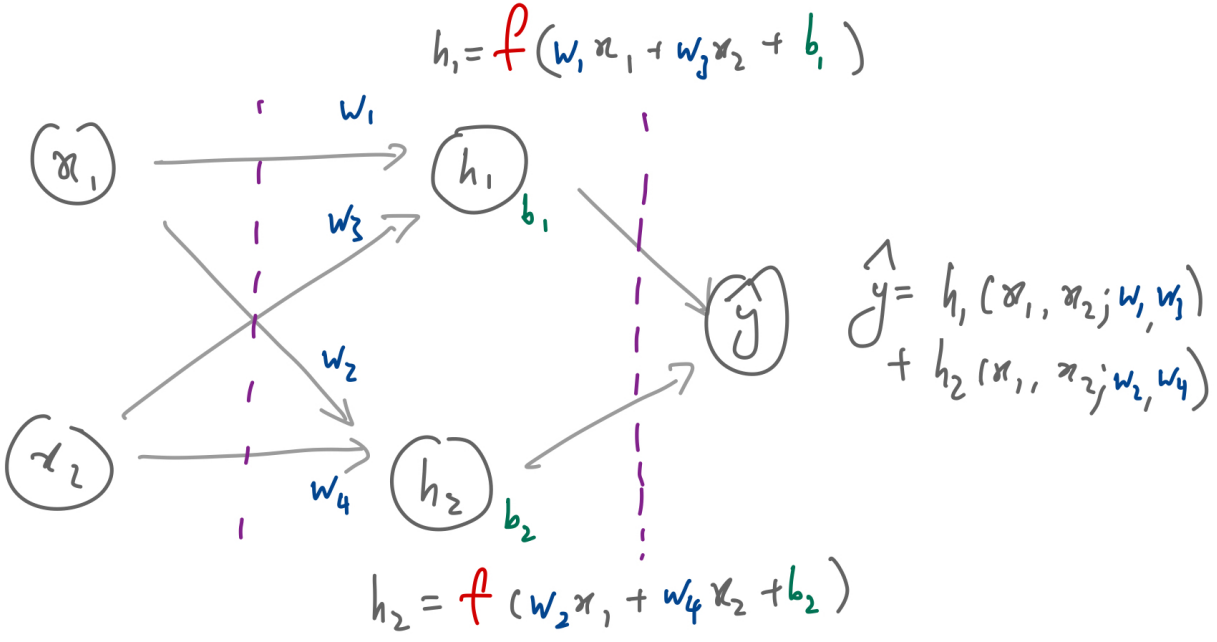


Figure: Made up example of a neural network (that you can code/train up yourself in principle; see notebook).

- one feed-forward results in $\hat{y}$

  $\rightarrow$ evaluate a loss function $J(Y, \hat{y})$

  $\rightarrow$ J depends on problem, e.g. one sided for classification, L^2 for regression, information entropy based etc.

Neural networks: training

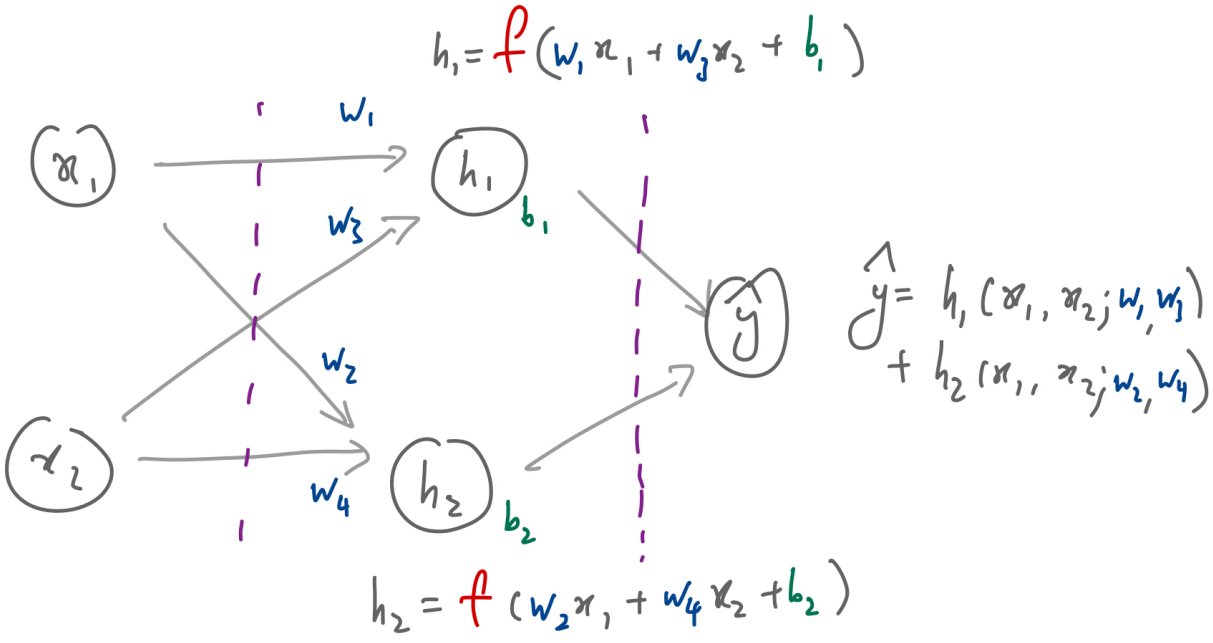


Figure: Made up example of a neural network (that you can code/train up yourself in principle; see notebook).

- aim: find minimum J by changing model parameters $\theta = (w_i, b_i)$

  $\rightarrow$ want to solve something like $\partial J / \partial \theta = 0$ (abusing notation here...)

Neural networks: training

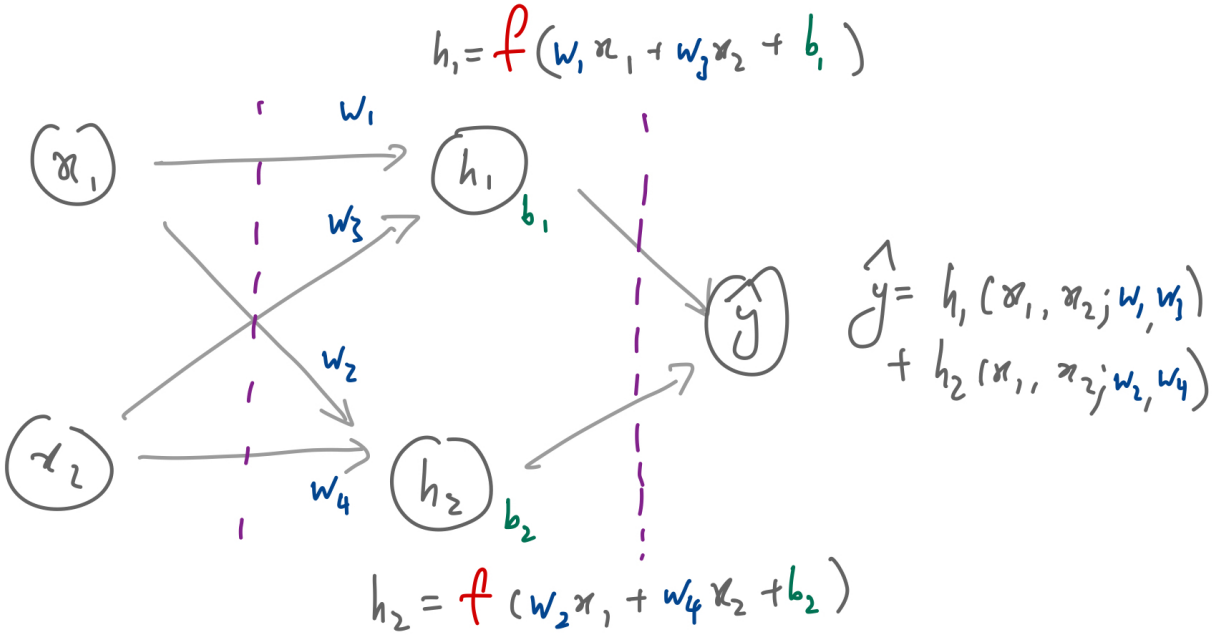


Figure: Made up example of a neural network (that you can code/train up yourself in principle; see notebook).

- back-propagation (chain rule basically): e.g.,

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial h_1}\frac{\partial h_1}{\partial w_1} = \frac{\partial J}{\partial h_1} f'(w_1 x_1 + \ldots) x_1.$$

$\rightarrow$ do for all, SGD etc. and iterate until convergence/bored

Neural networks: training

- more nodes?

  $\rightarrow$ more equations
- more layers?

  $\rightarrow$ longer chains
- activation functions?

  $\rightarrow$ choose piecewise differentiable and 'easy' functions
- other components within the hidden layer?

  (e.g. convolutional kernels in CNNS, hidden states in RNNs)

  $\rightarrow$ assume differentiability but proceed as usual

- constraint and/or dynamical layers?

  $\rightarrow$ can deal with in principle (e.g. adjoints), but beyond this course...

Perceptrons

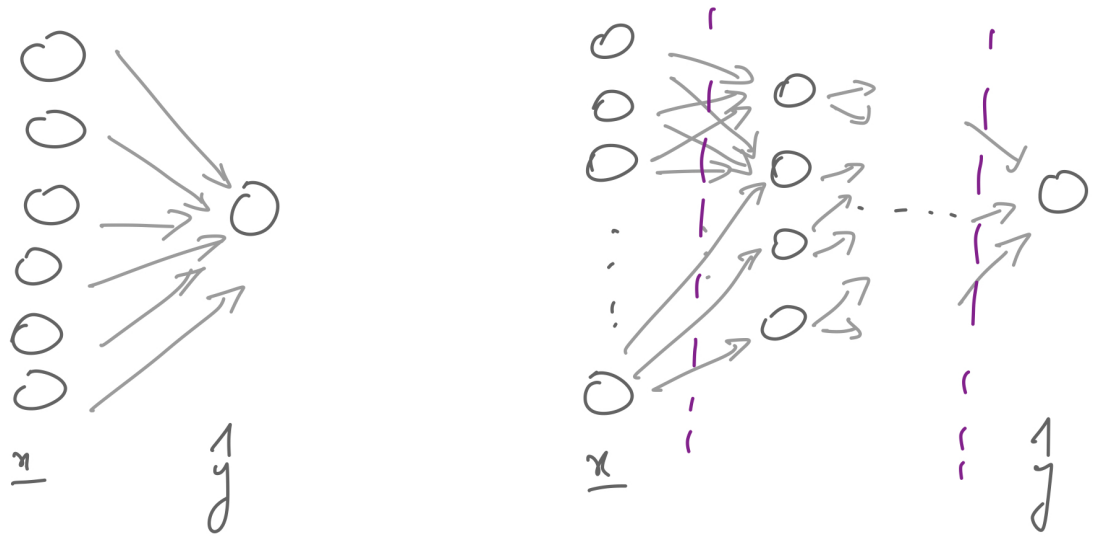


Figure: Schematic of perceptrons (no hidden layers) and MLPs (at least one hidden layer).

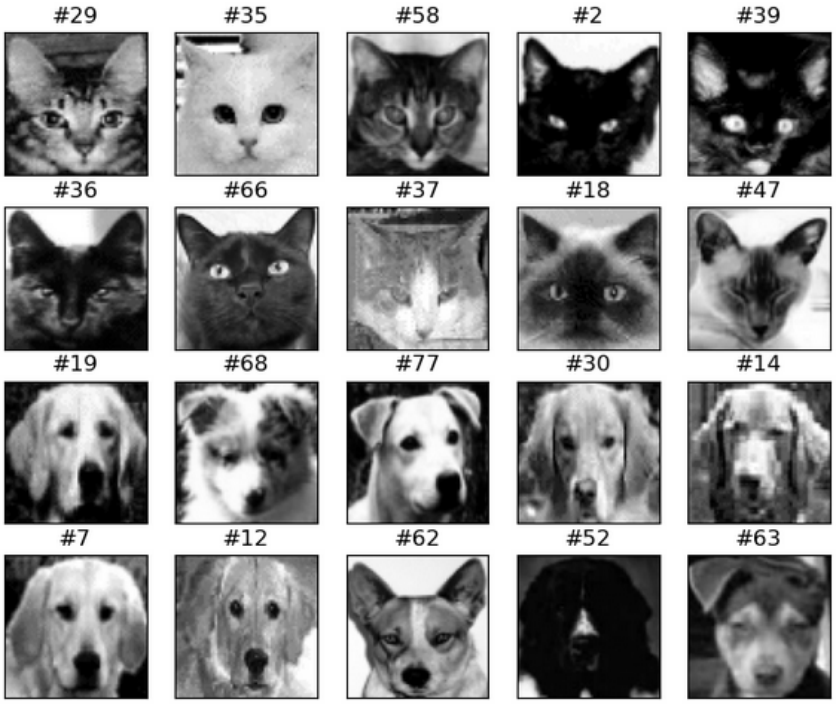
- perceptrons

  $\rightarrow$ no hidden layers, may have activation functions

  $\rightarrow$ can only really do binary classification (i.e. 'percept')

  $\rightarrow$ case of no activation function $\Rightarrow$ matrix inversion essentially

Perceptrons: classification



- classifying cats and dogs

  $\rightarrow$ this is a hard problem!

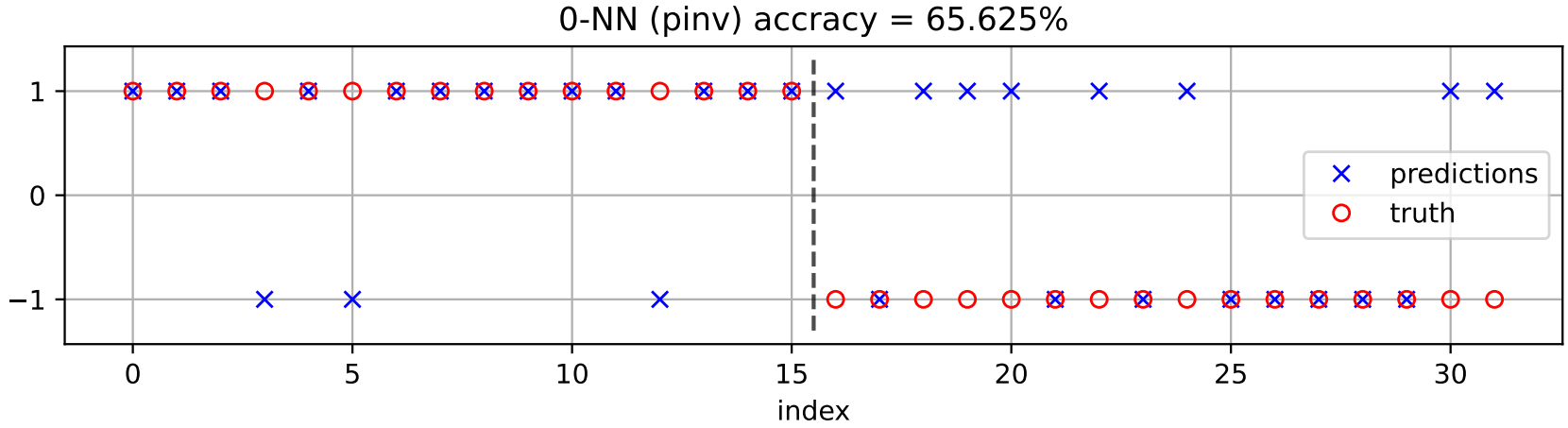
  $\rightarrow$ cats labelled as 1 and dogs as -1

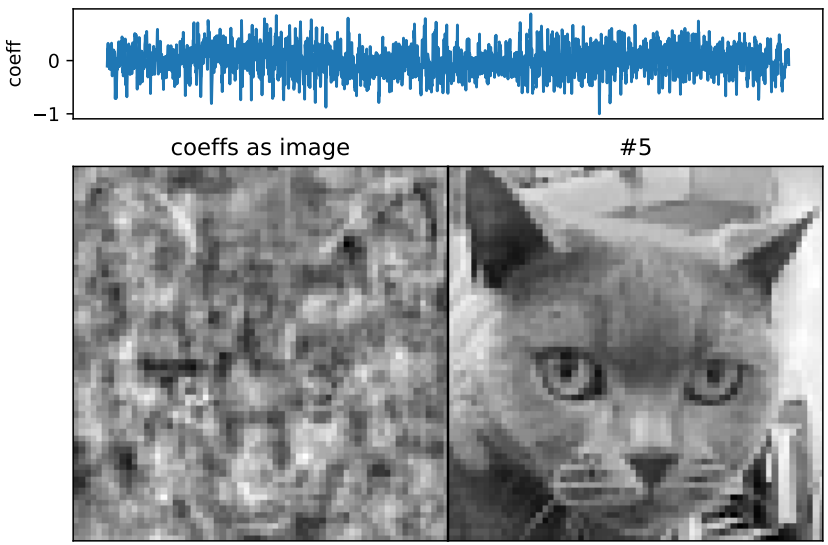
  $\rightarrow$ throw in full image

  $\rightarrow$ standardise, train/test split

  $\rightarrow$ can probe the model in this case, feature identification

Perceptrons: classification





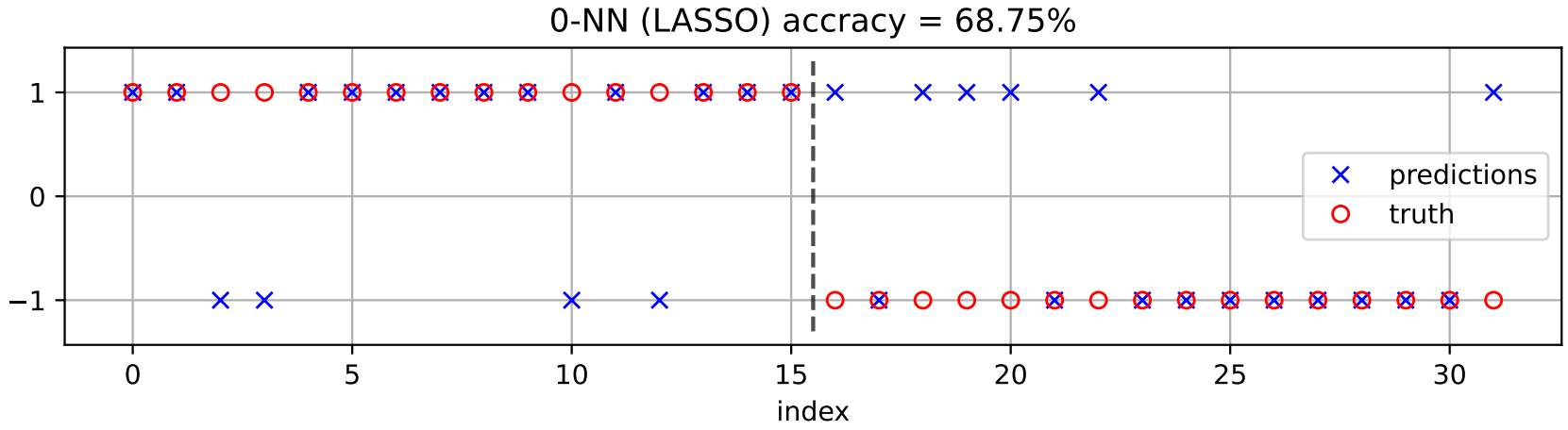
- just matrix inversion

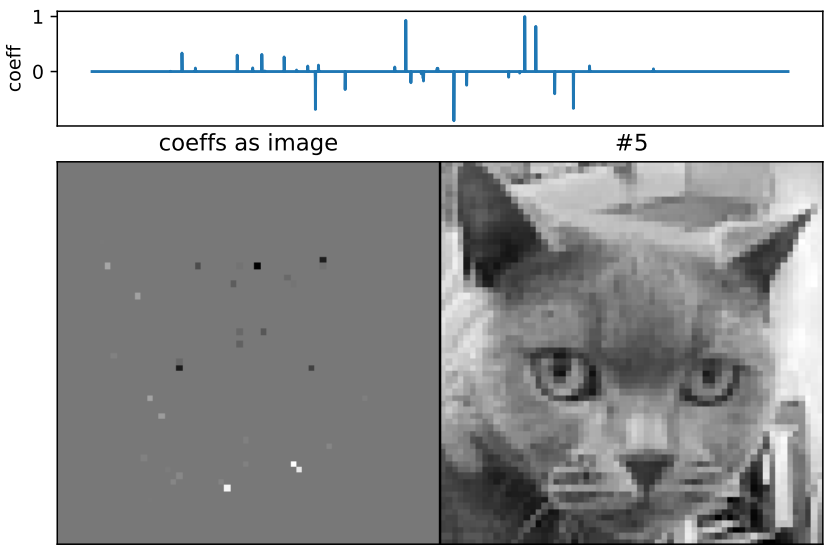
  $\rightarrow L^2$ minimisation with no penalisation

  $\rightarrow$ many non-zero values

  $\rightarrow$ some sort of 'shape', 'eyes' and 'facial features' identified?

Perceptrons: classification





- Lasso optimisation instead

  $\rightarrow L^2$ minimisation with L^1 penalisation

  $\rightarrow$ promotes sparsity (but α dependent)

  $\rightarrow$ 'eyes', 'forehead' and 'mouth' important?

MLPs

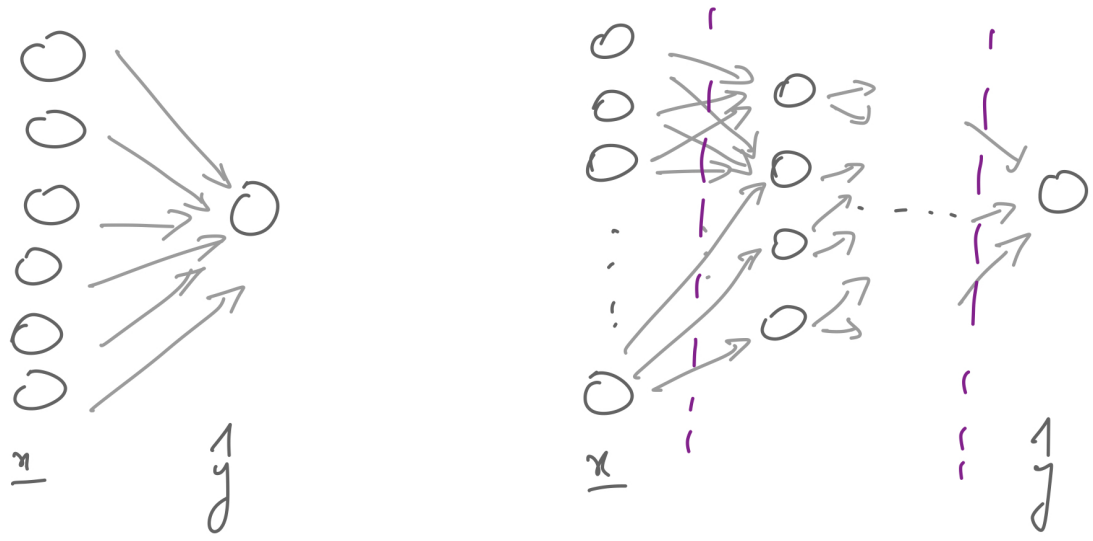


Figure: Schematic of perceptrons (no hidden layers) and MLPs (at least one hidden layer).

- **multi-layer** perceptrons (MLPs)

  $\rightarrow$ bit of a misnomer, because above limitation with perceptrons do not strictly apply

  $\rightarrow$ sometimes artificial neural networks (ANNs) (misleading?)

  $\rightarrow$ can be 'deep' neural networks (DNNs) (misleading?)

MLPs: classification

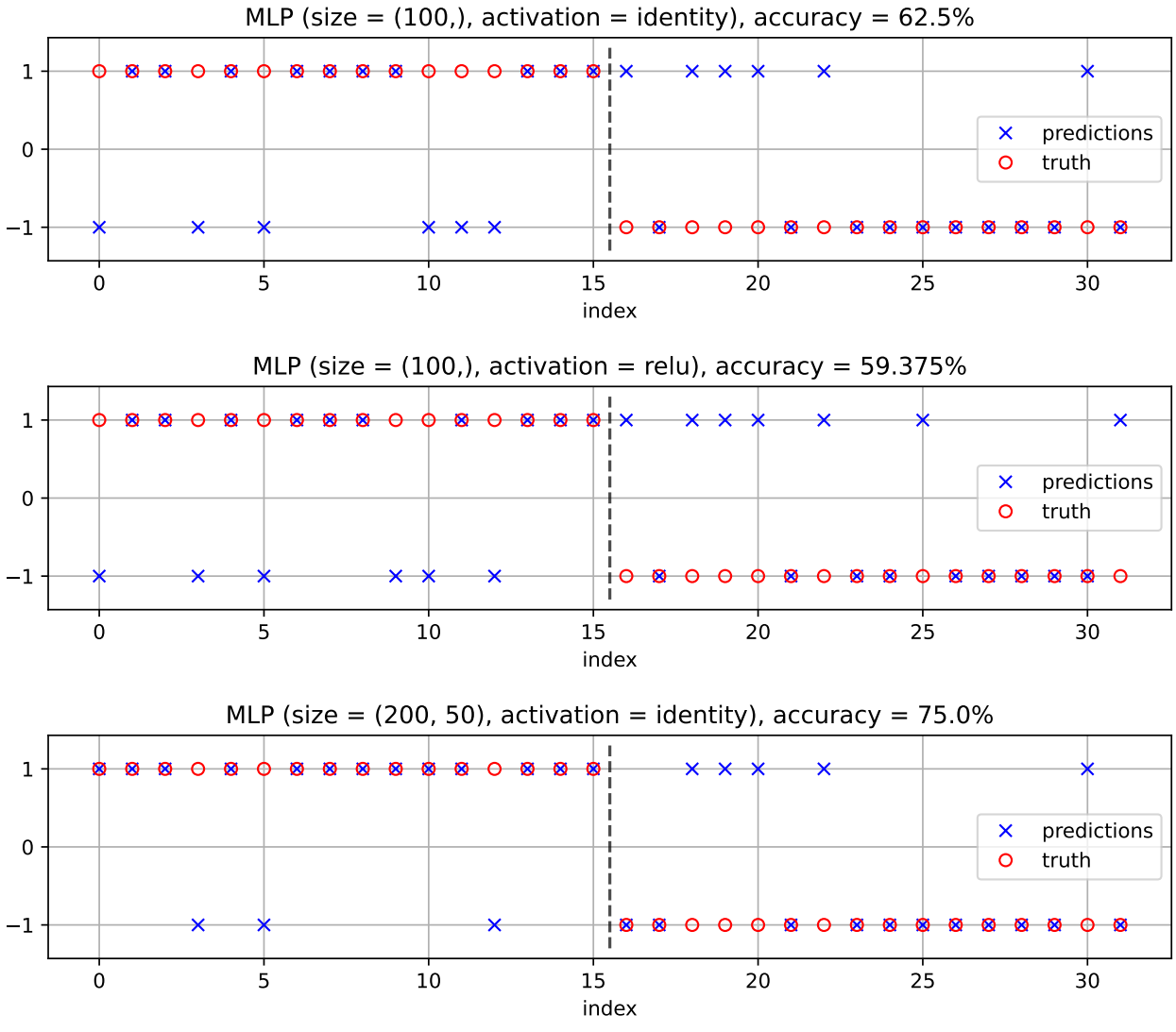


Figure: Three different attempts at using MLP to do full image classification.

MLPs: classification

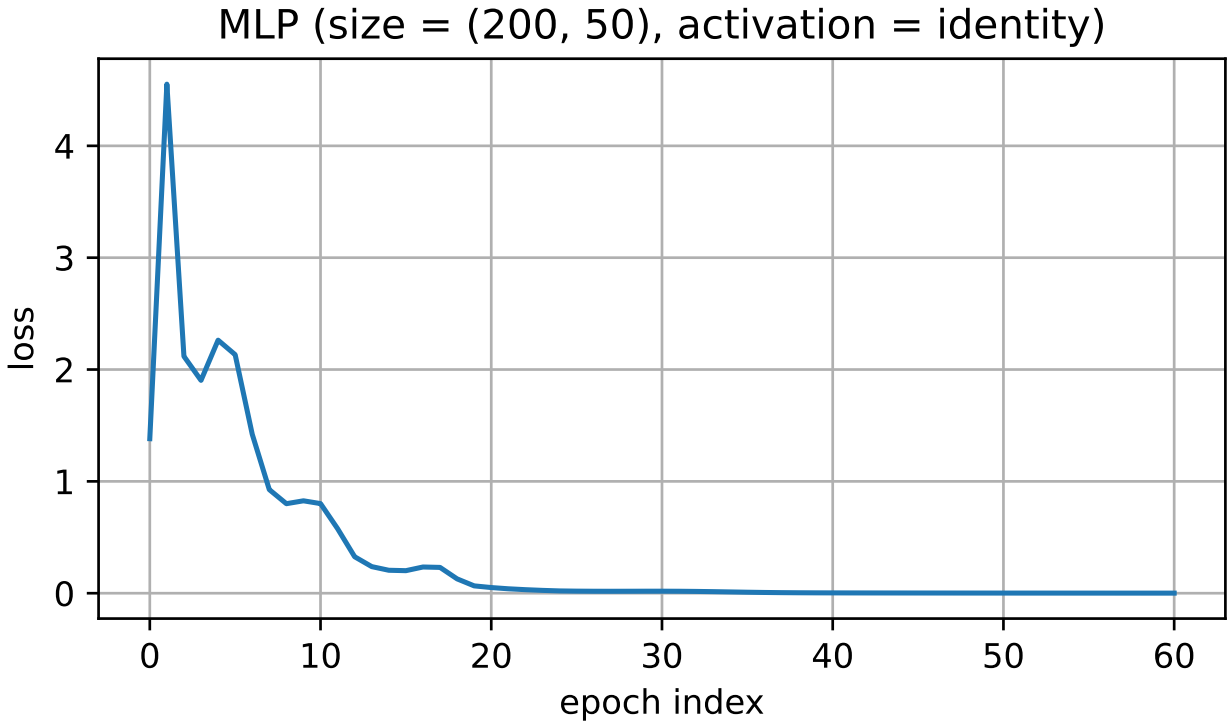


Figure: example of a loss curve with training epoch.

- an example of a loss curve with training decreasing with iterations (epoch)
  → continues until some criterion is reached
  → early-stopping is possible (but not considered here)

MLPs: regression problem

- just using cats
  $\rightarrow$ predict top half from bottom half
  $\rightarrow$ vice-versa might work better?

- again, this is a hard problem!

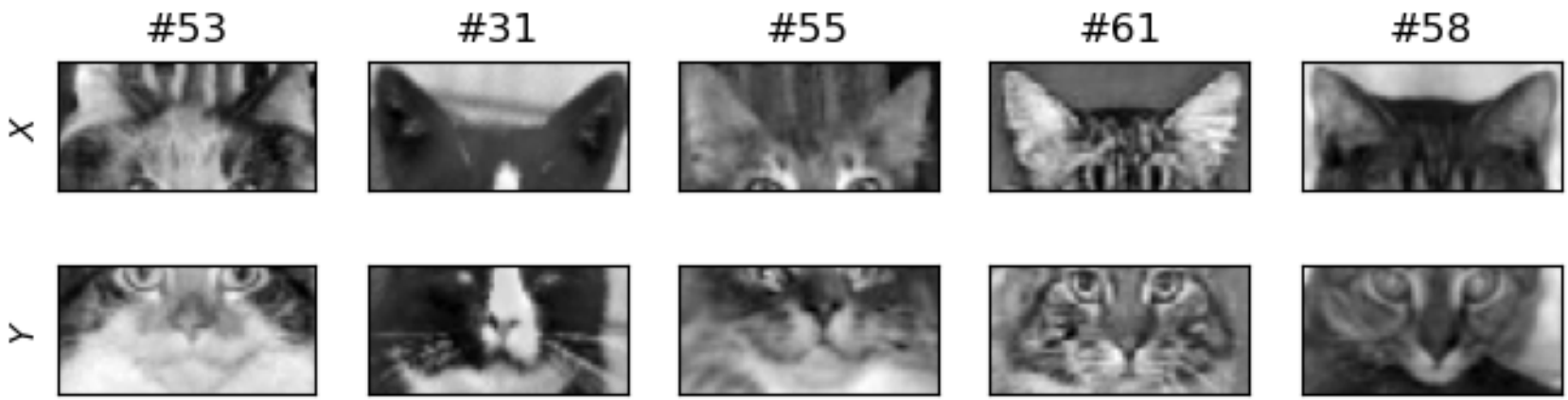


Figure: No animals were hurt in the present demonstration.

MLPs: regression problem

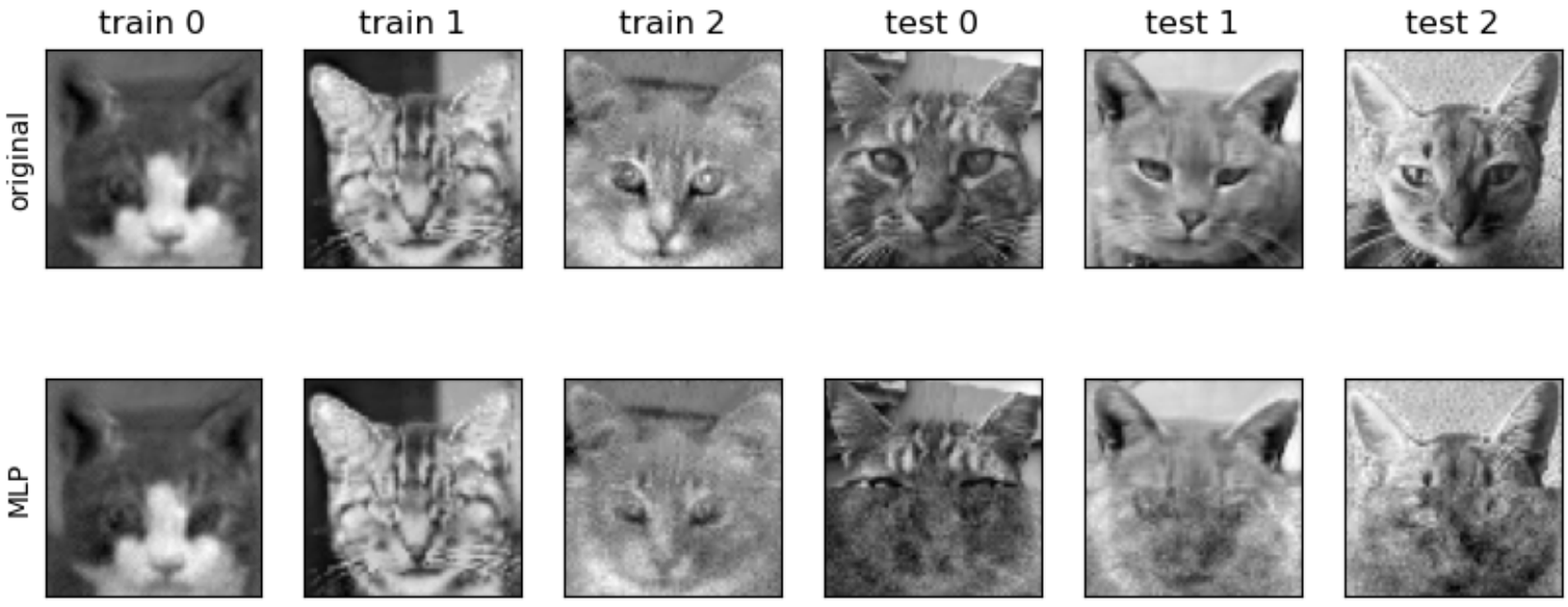


Figure: Predicting bottom half from top half, default settings.

- note that even the training set had issues doing to the prediction...

  $\rightarrow$ predictions can be quite cursed if eye happens to be in top half (e.g. grant us eyes)

MLPs: regression problem

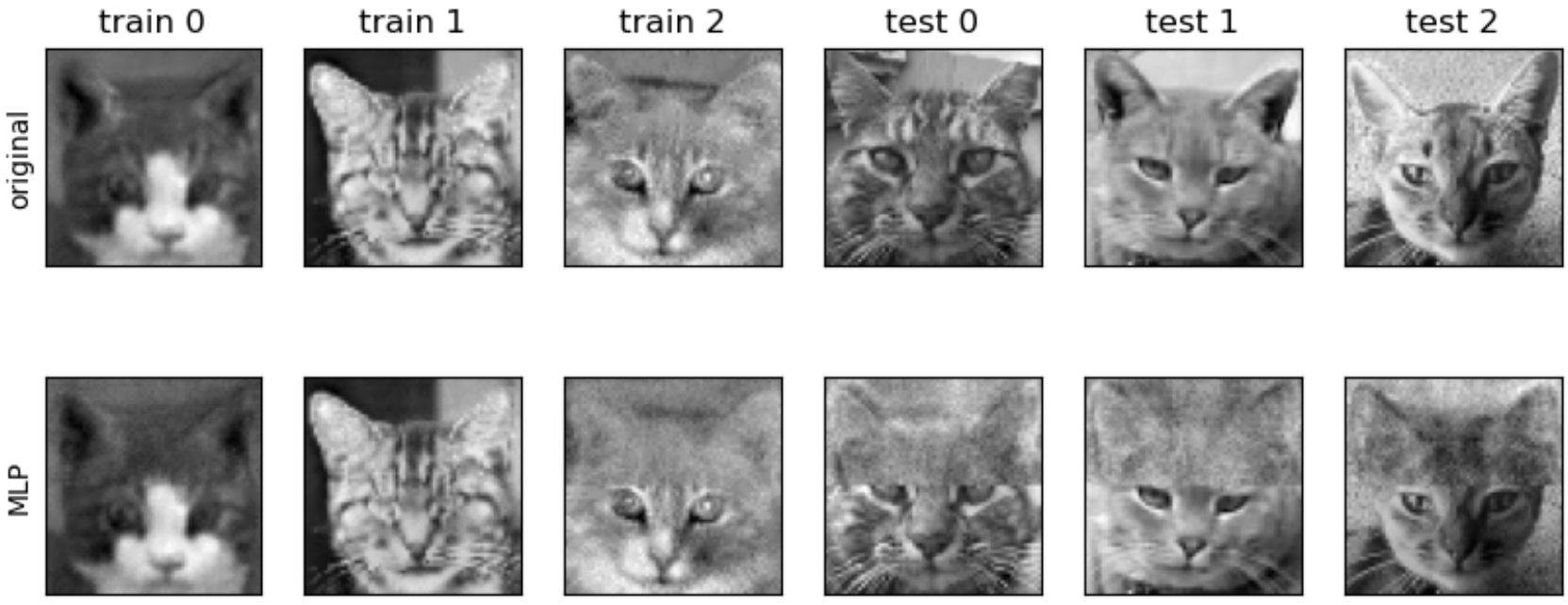


Figure: Predicting top half from bottom half, default settings.

- note that even the training set had issues doing to the prediction...

  $\rightarrow$ less cursed at least subjectively

  $\rightarrow$ at least somewhat continuous predictions?

MLPs: regression problem

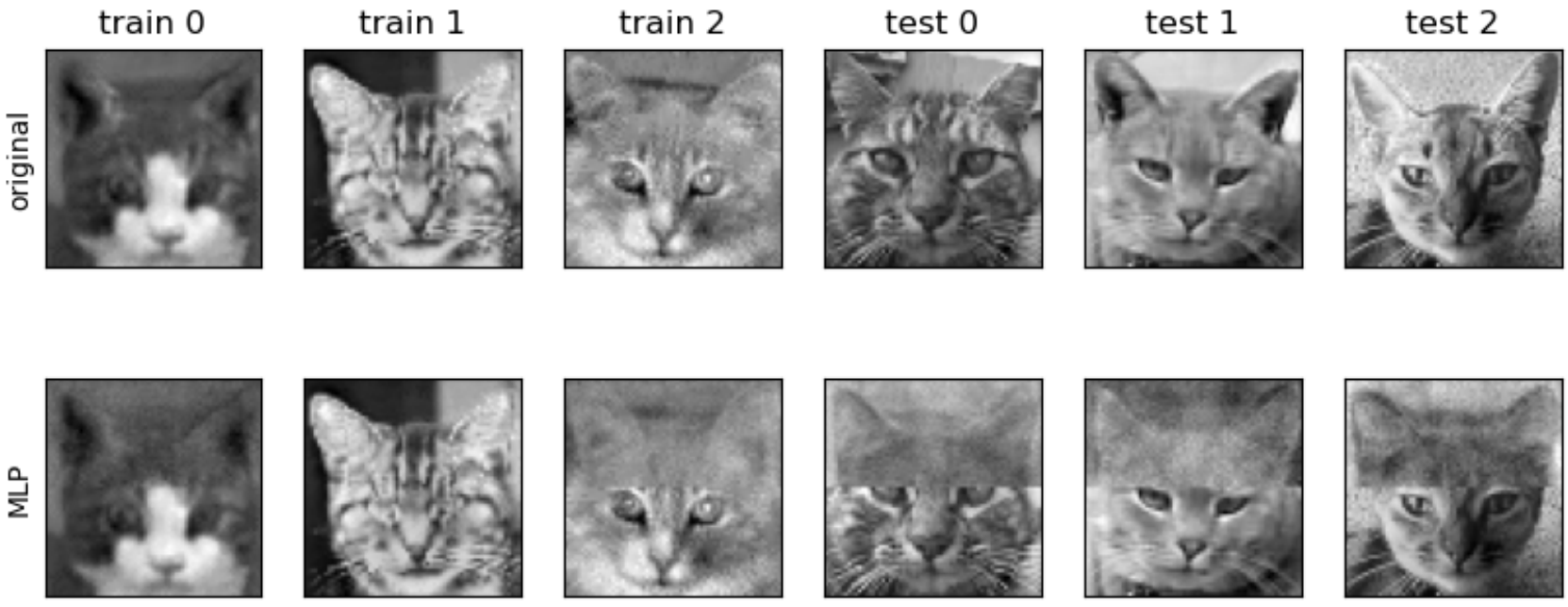


Figure: Predicting top half from bottom half, reasonably complicated.

- note that even the training set had issues doing to the prediction...

  $\rightarrow$ not doing much better with substantially increased complexity

Demonstration

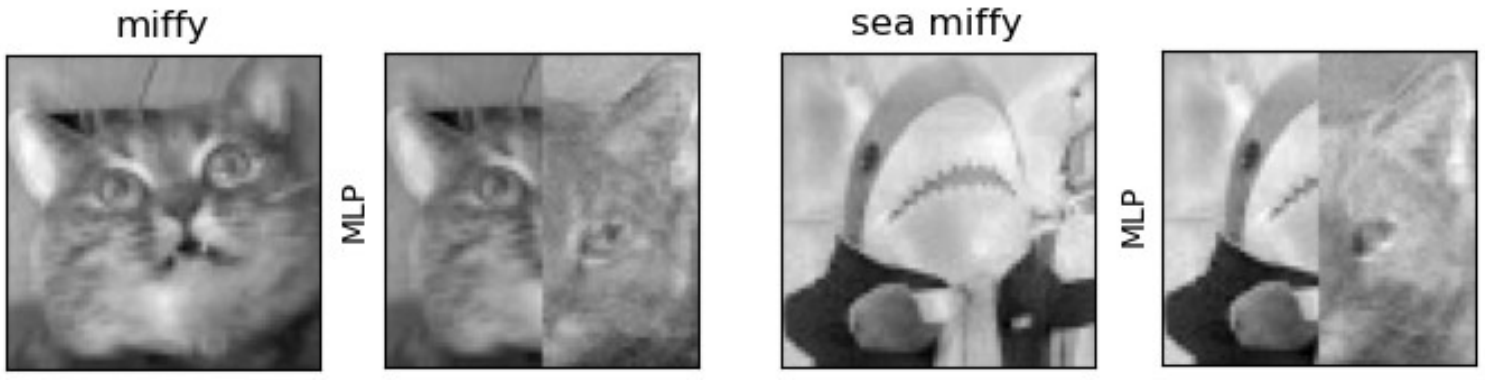


Figure: Cursed beasts reconstruction (out of sample application of MLP).

- idea behind neural networks
  $\rightarrow$ ways to control over-fitting not considered here (e.g. dropout, regularisation)
- note: MLPs are fully-connected by default, dimensionality blows up quickly!
- need to cross-validate and tune hyper-parameters accordingly!

Moving to a Jupyter notebook $\rightarrow$